

Teste de Software

Roteiro Sprint 5 - BDD como linguagem de cobertura semântica

Como ler este roteiro

Blocos azuis trazem conceito ou continuidade. Leia com atenção; não há resposta a registrar.

Blocos rosa trazem investigação técnica do aluno. **Você precisa registrar sua resposta no arquivo Word de entrega antes de avançar.**

Blocos amarelos trazem atenção técnica ou troubleshooting.

Blocos roxos trazem comandos, prompts ou esqueletos de código para adaptar. NUNCA cole o conteúdo bruto - você precisa preencher as partes em [COLCHETES] com seus próprios dados.

Blocos verdes trazem resultado esperado, autoavaliação ou reflexão final. Em ANP, são especialmente importantes - é como você sabe que está no caminho certo sem o professor ao lado.

Atenção: respostas sem ancoragem nos autores (RAHMAN et al., 2024; BSTQB, 2023; PAPADAKIS et al., 2019), sem ancoragem no cenário do internet banking, ou copiadas direto do roteiro sem adaptar aos seus dados do Sprint 4 não serão consideradas tecnicamente válidas.

Objetivo deste sprint

No Sprint 4 você descobriu, com o mutation testing, quais regiões do seu sistema a sua suíte de testes era cega - os mutantes sobreviventes que apareceram no relatório do mutmut. Mas duas perguntas ficaram em aberto: primeiro, dentro dessa lista de mutantes sobreviventes, há algum que nenhum teste possível poderia matar? Segundo, como comunicar essa lacuna para quem não programa - um gerente, um auditor, um analista de compliance?

São essas duas perguntas que o Sprint 5 responde: (1) como classificar criticamente os mutantes sobreviventes do Sprint 4 entre os que são equivalentes (não matáveis) e os que apenas revelam insuficiência da minha suíte; (2) como transformar os do segundo tipo em cenários de negócio legíveis, executáveis e auditáveis.

A ferramenta para (1) é um conceito teórico novo - mutante equivalente (PAPADAKIS et al., 2019, p. 286). A ferramenta para (2) é BDD (Behavior-Driven Development) com pytest-bdd. Você vai escrever cenários em Gherkin - uma linguagem estruturada em português que qualquer pessoa de

negócio entende - e em seguida implementar os passos (steps) em Python que executam esses cenários contra o seu sistema.

Antes de começar - pré-requisitos obrigatórios

Este sprint não funciona se faltar qualquer um dos itens abaixo. Confira cada um antes de prosseguir:

[] Sprint 4 completo, com o arquivo Word de entrega contendo a tabela do Passo 5 com a lista de mutantes sobreviventes registrados.

[] Pasta `internet_banking_tests` intacta do Sprint 4: `app.py` (com `config` externo), `config.py`, `conftest.py` atualizado, `test_ia_gerado.py`, `test_hypothesis.py`.

[] Ambiente virtual `venv` ativado.

[] `pytest-bdd` e `mutmut` instalados (Passo 1 cobre a instalação do `pytest-bdd` se ainda não estiver).

Passo 1. instale o `pytest-bdd` e crie a estrutura de pastas

Abra um terminal no VS Code, ative o ambiente virtual e instale o `pytest-bdd`:

```
# Windows
.\venv\Scripts\activate

# macOS / Linux
source venv/bin/activate

# Instalação
pip install pytest-bdd
```

Crie a estrutura de pastas dentro de `internet_banking_tests`:

```
# Dentro da pasta internet_banking_tests
mkdir features
mkdir features/steps
```

Resultado esperado: a pasta `internet_banking_tests` agora tem uma subpasta `features/` e dentro dela uma subpasta `steps/`.

Passo 2. resgate a lista bruta de mutantes sobreviventes do seu Sprint 4

Abra o seu arquivo Word de entrega do Sprint 4 e localize a tabela do Passo 5, onde você registrou os mutantes sobreviventes detectados pelo comando `mutmut results --surviving`.

Transcreva abaixo a lista bruta - apenas os dados que você já tinha lá, sem classificação adicional ainda. Se a sua lista tem mais de 4 mutantes, escolha 4 mais representativos. Se tem menos de 4, complete o que tiver e siga - o roteiro funciona com 2 mutantes obrigatórios e até 2 opcionais.

MUTANTE 1 (obrigatório):

Número (de mutmut show N):

Linha original do app.py:

Linha mutada (alteração introduzida pelo mutmut):

MUTANTE 2 (obrigatório):

Número:

Linha original do app.py:

Linha mutada:

MUTANTE 3 (opcional - ponto extra):

Número:

Linha original do app.py:

Linha mutada:

MUTANTE 4 (opcional - ponto extra):

Número:

Linha original do app.py:

Linha mutada:

Autoavaliação antes de avançar: se você consegue identificar para cada mutante exatamente qual caractere ou operador mudou (por exemplo: `<` virou `<=`, ou `==` virou `!=`, ou o número 0 virou 1), você está pronto para o Passo 3. Se algum dado está incompleto, abra novamente o Sprint 4 e complete antes de prosseguir.

Conceito novo do Sprint 5: o mutante equivalente

No Sprint 4, todos os mutantes sobreviventes da sua lista foram tratados igualmente - eram regiões do código que sua suíte não cobria. Mas há uma distinção crítica que vale a pena introduzir agora.

Mutante equivalente: é um mutante cuja alteração sintática produz código semanticamente idêntico ao original - ou seja, retorna exatamente o mesmo resultado para qualquer entrada possível. Nenhum teste poderá matá-lo, porque não há diferença observável de comportamento entre o original e o mutado (PAPADAKIS et al., 2019, p. 286).

Mutante por insuficiência da suíte: é um mutante que poderia ser morto - existe uma entrada que faria o código mutado produzir resultado diferente do código original - mas a sua suíte de testes simplesmente não tem nenhum teste que execute essa entrada.

Exemplos concretos no internet banking:

Equivalente: se o `app.py` original tem `if conta is None:` e o `mutmut` produz `if conta == None:` - são duas formas equivalentes de testar `None` em Python. Nenhum teste pode diferenciar.

Insuficiência da suíte: se o `app.py` original tem `if conta_origem["saldo"] < valor:` e o `mutmut` produz `if conta_origem["saldo"] <= valor:` - são semanticamente diferentes. Existe uma entrada que distingue: quando saldo é exatamente igual a valor. Se a sua suíte não tem teste para essa borda específica, o mutante sobrevive não por equivalência, mas por insuficiência.

Importante: identificar automaticamente se um mutante é equivalente é um problema indecidível em sua forma geral (PAPADAKIS et al., 2019, p. 286). Por isso a classificação requer análise humana - é parte do trabalho do testador, não do `mutmut`.

Passo 3. classifique individualmente cada mutante da sua lista

Para cada mutante que você transcreveu no Passo 2, analise a alteração sintática e decida: ela produz comportamento semanticamente diferente do original (insuficiência da suíte) ou semanticamente idêntico (equivalente)?

Use o procedimento de três perguntas abaixo para cada mutante, na ordem:

Procedimento de classificação - para cada mutante:

Pergunta 1: Existe alguma entrada (valor de origem, destino, valor, conta) para a qual o código original retornaria um resultado diferente do código mutado?

Pergunta 2: Se sim - qual entrada concreta diferenciaria? Descreva em termos do internet banking.

Pergunta 3: Se sua resposta à Pergunta 1 foi sim e você conseguiu responder a Pergunta 2 com uma entrada concreta - classifique como INSUFICIÊNCIA DA SUÍTE. Se sua resposta à Pergunta 1 foi não, ou se você não consegue descrever uma entrada diferenciadora - classifique como EQUIVALENTE.

Agora aplique o procedimento a cada um dos seus mutantes do Passo 2. Preencha:

CLASSIFICAÇÃO DO MUTANTE 1:

Resposta à Pergunta 1 (sim/não):

Resposta à Pergunta 2 (entrada diferenciadora, se aplicável):

Classificação final (EQUIVALENTE ou INSUFICIÊNCIA DA SUÍTE):

CLASSIFICAÇÃO DO MUTANTE 2:

Resposta à Pergunta 1:

Resposta à Pergunta 2:

Classificação final:

CLASSIFICAÇÃO DO MUTANTE 3 (opcional):

Resposta à Pergunta 1:

Resposta à Pergunta 2:

Classificação final:

CLASSIFICAÇÃO DO MUTANTE 4 (opcional):

Resposta à Pergunta 1:

Resposta à Pergunta 2:

Classificação final:

Autoavaliação antes de avançar: você precisa ter pelo menos 1 mutante classificado como INSUFICIÊNCIA DA SUÍTE entre os obrigatórios (MUTANTES 1 e 2) para prosseguir. Se ambos foram classificados como EQUIVALENTE, releia o conceito - é raro mas possível. Nesse caso, volte ao seu Sprint 4 e selecione outros 2 mutantes que mostrem comportamento diferenciável.

Passo 4. descreva os mutantes de insuficiência em termos de negócio

Os mutantes que você classificou como INSUFICIÊNCIA DA SUÍTE são os candidatos para virar cenários BDD nos próximos passos. Mas antes de escrever Gherkin, você precisa traduzir cada um para linguagem de negócio - termos que um gerente do banco entenderia.

Para cada mutante de insuficiência, preencha:

MUTANTE 1 (apenas se classificado como INSUFICIÊNCIA DA SUÍTE):

- a) Que regra de negócio do internet banking esta linha implementa?
- b) Qual entrada concreta (saldo, valor, conta) faria o sistema com a linha mutada se comportar diferente do sistema com a linha original?
- c) Como o sistema deveria responder a essa entrada (status HTTP, mensagem, saldo final)?

MUTANTE 2 (apenas se classificado como INSUFICIÊNCIA DA SUÍTE):

- a) Que regra de negócio do internet banking esta linha implementa?
- b) Qual entrada concreta diferenciaria os comportamentos?
- c) Como o sistema deveria responder a essa entrada?

Autoavaliação antes de avançar: se você consegue responder o item (a) em termos como "rejeitar transferência com saldo insuficiente" ou "recusar valor não positivo" - sem mencionar nomes de variáveis ou operadores Python - você está pronto para o Passo 5. Se a resposta saiu em linguagem técnica, reformule pensando em como um gerente descreveria a regra.

Passo 5. escreva os cenários Gherkin no arquivo .feature

Gherkin tem uma estrutura fixa - quatro palavras-chave em português: Dado (estado inicial), Quando (ação), Então (resultado esperado), E (continuação). Cada cenário descreve um caminho de comportamento do sistema.

No VS Code, clique direito sobre a pasta features/ → New File → digite transferencia.feature → Enter.

Cole o esqueleto abaixo no arquivo. Em seguida, substitua os blocos em [COLCHETES] usando os dados que você preencheu no Passo 4. Cada cenário deve traduzir uma das fichas do Passo 4 em formato Gherkin. Não há gabarito - cada aluno terá um arquivo diferente, porque cada um partiu de mutantes diferentes do Sprint 4.

```
# features/transferencia.feature
```

```
# language: pt
```

```
Funcionalidade: [nome da funcionalidade do internet banking  
que os mutantes do Passo 4 expuseram  
como insuficientemente testada]
```

```
Como cliente do internet banking  
Quero [o que o cliente quer fazer]  
Para que [qual benefício ou garantia o cliente obtém]
```

```
Contexto:
```

```
Dado que a conta 1 possui saldo de 1000.00  
E que a conta 2 possui saldo de 500.00
```

```
Cenario: [título em termos de negócio - vindo do MUTANTE 1]  
Quando [a ação que executa a entrada do Passo 4 item b]  
Entao [resultado esperado - do Passo 4 item c]  
E [verificação adicional, se aplicável]
```

```
Cenario: [título em termos de negócio - vindo do MUTANTE 2]  
Quando [...]  
Entao [...]
```

Atenção: a primeira linha # language: pt é obrigatória - é ela que diz ao pytest-bdd para reconhecer Funcionalidade, Cenário, Dado, Quando, Então em português. Se omitir, o parser usa inglês por padrão e nenhum cenário é reconhecido.

Autoavaliação antes de avançar: releia seus cenários e responda mentalmente - se um gerente do banco lesse este arquivo, ele entenderia o que o sistema faz? Se a resposta for não, reescreva os títulos e as ações usando termos de negócio em vez de termos técnicos.

Passo 6. use o Copilot para gerar features/steps/transferencia_steps.py

Os steps são as funções Python que executam cada linha do .feature. No VS Code, crie o arquivo features/steps/transferencia_steps.py vazio. Abra o GitHub Copilot Chat (Cmd+Shift+I no macOS / Ctrl+Alt+I no Windows). Cole o prompt abaixo, substituindo [SEU_FEATURE] pelo conteúdo do .feature que você acabou de escrever no Passo 5:

Prompt para o Copilot - adapte ao seu .feature:

Gere o conteúdo de features/steps/transferencia_steps.py para implementar os steps pytest-bdd do arquivo .feature abaixo, aplicado ao sistema de internet banking da disciplina de Teste de Software.

Arquivo .feature:
[SEU_FEATURE]

Requisitos arquiteturais obrigatórios (Sprint 4 estabeleceu esta arquitetura):

- Use Flask test client por importação direta: from app import app
- NÃO use requests HTTP - o servidor Flask não estará rodando
- NÃO redefina fixture autouse de reset do banco - ela já existe no conftest.py da raiz da pasta
- Use a fixture client do conftest.py existente: def step(client, ...)
- Use escenarios("../transferencia.feature") para carregar o .feature
- Use @given, @when, @then com parsers.parse para extrair valores dos cenários
- Para o estado entre passos (resposta HTTP, etc.), use uma fixture pytest com escopo de função

Gere apenas o código Python. Sem explicações adicionais.

Por que esses requisitos são obrigatórios: o Sprint 4 estabeleceu que o sistema opera por Flask test client com configuração externalizada via config.py - sem servidor HTTP externo, sem fixture duplicada. O Copilot foi treinado em tutoriais BDD genéricos que usam requests e fixtures locais. Você precisa revisar o código gerado contra os três critérios acima antes de aceitar.

Salve o código gerado no arquivo features/steps/transferencia_steps.py.

Passo 7. revise criticamente o código gerado pelo Copilot

Antes de executar, abra o `transferencia_steps.py` gerado pelo Copilot e responda:

Investigação Técnica - revisão crítica:

- a) O Copilot usou Flask test client (`from app import app + app.test_client()`) ou tentou usar requests HTTP? Se usou requests, registre a linha e explique por que isso violaria a arquitetura do Sprint 4:
- b) O Copilot redefiniu alguma fixture de reset de banco no `transferencia_steps.py`? Se sim, registre a linha. Por que essa redefinição entraria em conflito com o `confest.py` do Sprint 4?
- c) O Copilot acertou o uso da fixture client do `confest`? Se não, qual ajuste você precisa fazer manualmente?

Sustente em COUTINHO; NASCIMENTO (2025): o Copilot foi treinado em milhares de tutoriais BDD que usam padrão genérico. Mas o seu projeto não é genérico - tem arquitetura germinada nos Sprints 1 a 4 que precisa ser respeitada. Aceitar o que o Copilot gera sem revisão crítica quebra a continuidade arquitetural.

Passo 8. ajuste o código gerado e execute os cenários

Com base na sua revisão do Passo 7, faça os ajustes necessários no `transferencia_steps.py`. Em seguida, execute os cenários:

```
pytest features/ -v
```

Resultado esperado:

O `pytest-bdd` coleta e executa os cenários do seu `transferencia.feature`. Cada cenário aparece como um teste individual. Os cenários relativos aos seus mutantes de insuficiência devem PASSAR sobre o `app.py` original.

Se algum cenário falhar: (i) revise se o cenário está descrevendo o comportamento correto do sistema original; (ii) revise se os steps estão executando a ação correta; (iii) revise se a primeira linha `# language: pt` está presente no `.feature`.

Passo 9. valide empiricamente se seus cenários matam os mutantes do Sprint 4

Aqui está a ponte epistemológica do Sprint 5. Você escreveu cenários Gherkin partindo da hipótese de que matariam os mutantes que classificou como INSUFICIÊNCIA DA SUÍTE. É hora de testar essa hipótese.

Rode o mutmut novamente, agora com a suíte ampliada - o `test_flask_client.py` do Sprint 4 mais a pasta `features/` do Sprint 5:

```
# Limpe o cache do mutmut antes de medir novamente
rm -rf .mutmut-cache mutants

# Reexecute o mutmut sobre o app.py
mutmut run app.py

# Examine os resultados
mutmut results

# Verifique especificamente os mutantes que você quis matar
mutmut show [NÚMERO DO MUTANTE 1]
mutmut show [NÚMERO DO MUTANTE 2]
```

Registre os resultados:

MUTANTE 1 - status após Sprint 5 (SURVIVED ou KILLED):

MUTANTE 2 - status após Sprint 5 (SURVIVED ou KILLED):

Análise crítica do resultado:

Cenário ideal: os mutantes que você classificou como INSUFICIÊNCIA DA SUÍTE no Passo 3 mudaram de SURVIVED para KILLED após adição dos cenários Gherkin. Isso valida sua classificação e mostra que BDD efetivamente cobre lacunas semânticas que o `test_ia_gerado.py` não cobria.

Se algum mutante de insuficiência continuou SURVIVED: seu cenário Gherkin não estava verificando especificamente o que difere entre o código original e o código mutado. Volte ao Passo 4 e refine a entrada diferenciadora do item (b); depois ajuste o Passo 5 e tente novamente.

Se algum mutante que você classificou como EQUIVALENTE foi morto inadvertidamente: sua classificação do Passo 3 estava errada - o mutante era de insuficiência, não equivalente. Registre isso na sua reflexão final.

Tabela epistemológica - as três métricas de qualidade da sua suíte

Você agora tem três métricas complementares que medem dimensões diferentes da qualidade da sua suíte. Cada uma responde a uma pergunta distinta - nenhuma substitui as outras.

Métrica	Pergunta que ela responde	Sprint em que você produziu
Cobertura de linhas	O pytest executou esta linha do app.py?	Sprint 2 (pytest-cov com term-missing)
Mutation score	O pytest perceberia se esta linha estivesse errada?	Sprint 4 (mutmut sobre app.py)
Cobertura semântica	Um leitor de negócio entenderia o que esta linha precisa garantir?	Sprint 5 (cenários Gherkin em features/transferencia.feature)

Reflexão do Sprint 5 - registre em seu arquivo Word antes de avançar

Responda com suas próprias palavras. Não existe resposta certa. O que importa é que o texto revele o seu raciocínio real durante a execução.

1. Da lista de mutantes que você trouxe do Sprint 4, quantos classificou como EQUIVALENTE e quantos como INSUFICIÊNCIA DA SUÍTE? Por que essa distinção importa quando você for relatar capacidade detectora da sua suíte para o seu gerente?
2. Compare o arquivo features/transferencia.feature que você produziu neste sprint com o arquivo test_ia_gerado.py do Sprint 1. Para cada um, em uma frase: quem conseguiria ler e entender? O que isso significa na prática para o internet banking?

3. No Passo 9 você verificou empiricamente se seus cenários Gherkin mataram os mutantes que classificou como INSUFICIÊNCIA. Algum continuou SURVIVED apesar do seu cenário? Algum classificado como EQUIVALENTE foi morto inadvertidamente? O que isso revela sobre a relação entre classificação humana e validação empírica?
4. Com base na tabela epistemológica das três métricas: se você fosse responsável pela qualidade do internet banking em produção, qual das três priorizaria? Use uma frase para defender.
5. Sustente em RAHMAN et al. (2024, p. 55-57): BDD substitui os testes dos sprints anteriores ou os complementa? Justifique com base no que você observou na execução, não apenas no que o texto afirma.
6. Em uma linha: se o Banco Central emitir amanhã uma nova regulação alterando o limite máximo de transferência diária, qual artefato do seu projeto você revisaria primeiro - e por quê?

Entrega no AVA

Arquivo Word único contendo, na ordem:

- (1) Lista bruta dos mutantes do Sprint 4 transcrita no Passo 2 (MUTANTES 1 e 2 obrigatórios; 3 e 4 valem ponto extra).
- (2) Classificação dos mutantes (Passo 3) - resposta às três perguntas e classificação final EQUIVALENTE/INSUFICIÊNCIA.
- (3) Descrição em termos de negócio (Passo 4) para os mutantes de insuficiência.
- (4) Conteúdo completo do features/transferencia.feature (copie e cole).
- (5) Revisão crítica do código do Copilot (Passo 7).
- (6) Conteúdo completo do features/steps/transferencia_steps.py após ajustes.
- (7) Print do terminal mostrando pytest features/ -v bem-sucedido (Passo 8).
- (8) Print do mutmut results após o Passo 9 e registro dos status dos MUTANTES 1 e 2.
- (9) Reflexão final do Sprint 5 (6 perguntas).

Pontuação: até 8 pontos pelos itens obrigatórios (MUTANTES 1 e 2), até 2 pontos extras pelos opcionais (MUTANTES 3 e 4). Total possível: 10 pontos.

Critérios de avaliação: (i) coerência entre os mutantes do Sprint 4, a classificação do Passo 3 e os cenários Gherkin do Passo 5; (ii) ancoragem em RAHMAN et al. (2024), COUTINHO; NASCIMENTO (2025), PAPADAKIS et al. (2019), BSTQB (2023); (iii) revisão crítica honesta do código do Copilot - reconhecer ajustes necessários vale mais que esconder problemas; (iv) execução empírica bem-sucedida do mutmut comprovando o resultado da classificação; (v) profundidade da reflexão epistemológica.

Referências Bibliográficas

BSTQB - BRAZILIAN SOFTWARE TESTING QUALIFICATIONS BOARD. *Certified Tester Foundation Level Syllabus*: versão 4.0. [S. l.]: ISTQB; BSTQB, 2023. Capítulo 6, p. 52-53. Disponível em: https://www.bstqb.online/files/syllabus_ctfl_4.0br.pdf. Acesso em: maio 2026.

COUTINHO, N. de M.; NASCIMENTO, E. L. B. do. Desafios e benefícios da implementação de testes automatizados em empresas de software. *Cuadernos de Educación y Desarrollo*, v. 17, n. 4, e8221, p. 1-19, 2025. DOI: <https://doi.org/10.55905/cuadv17n4-176>. Disponível em: <https://ojs.cuadernoseducacion.com/ojs/index.php/ced/article/view/8221>. Acesso em: maio 2026.

PAPADAKIS, M.; KINTIS, M.; ZHANG, J.; JIA, Y.; LE TRAON, Y.; HARMAN, M. Mutation testing advances: an analysis and survey. *Advances in Computers*, v. 112, p. 275-378, 2019. DOI: 10.1016/bs.adcom.2018.03.015. Disponível em: <https://www.sciencedirect.com/science/article/abs/pii/S0065245818300305>. Acesso em: maio 2026.

RAHMAN, Md. S. et al. Evaluating the impact of test-driven development on software quality enhancement. *International Journal of Mathematical Sciences and Computing*, v. 10, n. 3, p. 51-76, 2024. DOI: 10.5815/ijmsc.2024.03.05. Disponível em: <https://www.mecs-press.org/ijmsc/ijmsc-v10-n3/IJMSC-V10-N3-5.pdf>. Acesso em: maio 2026.

PYTEST-BDD. *pytest-bdd documentation*. Disponível em: <https://pytest-bdd.readthedocs.io/en/latest/>. Acesso em: maio 2026.

CUCUMBER. *Gherkin Reference*. Disponível em: <https://cucumber.io/docs/gherkin/reference/>. Acesso em: maio 2026.